Prueba técnica

Este documento reúne las respuestas escritas de los módulos 1, 3, 4 y 5. El Módulo 2 es práctico: se entrega el enlace de GitHub, no el código como archivo o carpeta. Las mismas respuestas están en archivos Markdown (.md), accesibles desde el README de la raíz del repositorio.

Repositorio GitHub — Módulo 2: https://github.com/jeison-nura/pruebatecnicab. El repositorio incluye también el UML de entidades en modulo2_uml_entidades.md y la colección Postman en polizas-api/postman_collection.json.

Módulo 1 — Diseño de Sistema (System Design)

Caso: Plataforma de gestión de pólizas de arrendamiento (individuales y colectivas).

1. Arquitectura de alto nivel

Se propone un monolito modular sobre Spring Boot, organizado en módulos internos con comunicación por eventos, con integración asíncrona al CORE legado mediante el patrón Outbox, y un borde único (API Gateway) que centraliza seguridad y versionamiento.

Justificación de la forma de despliegue. Es una primera implementación con equipo pequeño y alcance acotado: un monolito modular da la velocidad de un solo despliegue (sin orquestación de múltiples servicios) y, al mismo tiempo, fuerza límites de módulo claros (Pólizas, Riesgos, Notificaciones, Integración CORE) que hacen la migración a microservicios evolutiva y no un rewrite. La disponibilidad 24/7 se resuelve en esta fase con réplicas stateless detrás de un balanceador; el estado vive en la base de datos y en el outbox, nunca en memoria del proceso.

Los flujos de la plataforma:

1. Flujo síncrono (borde): el frontend consume la API REST (`/v1/polizas`, `/v1/riesgos`).

Toda operación de escritura valida reglas de negocio, persiste el cambio y el evento de sincronización en la misma transacción, y responde al cliente sin esperar al CORE.

2. Flujo asíncrono (interior): un worker entrega los eventos del outbox al servicio

agnóstico de edición (capa media WebLogic) con reintentos, idempotencia, circuit breaker y DLQ. La entrega "sí o sí" se garantiza con tracking y reintentos, no con latencia acoplada.

3. Flujo de notificación: creación y renovación de pólizas emiten eventos que un

worker de notificaciones consume para despachar correo/SMS, con tracking de entrega en base de datos y reintentos ante fallo del proveedor.

4. Renovación automática: un scheduler (cronjob) escanea pólizas próximas a vencer,

aplica el incremento del IPC sobre el canon, recalcula la prima, cambia el estado a `RENOVADA` y emite los eventos correspondientes. Con réplicas, el job usa lock de líder (ShedLock) para ejecutarse una sola vez.

Decisión de migración declarada: si la aplicación crece, la evolución natural es microservicios + colas de mensajes + DLQ + workers por dominio. El monolito modular con puertos abstraídos hace ese paso incremental: cada módulo ya tiene su borde definido.

Ruta de extracción (por qué el diseño la abarata): los módulos viven en schemas separados sin joins ni FKs cruzados, y los vínculos entre módulos son por API/eventos (con correlationId en el payload, no por FK). Extraer un módulo es entonces un procedimiento operativo, no un rediseño: (1) `pg_dump` del schema del módulo y restore en su propia BD; (2) repoint de credenciales del worker/adapter correspondiente; (3) swap del event bus interno (un puerto) por el adapter de Kafka; (4) verificación con reconciliación y contract tests — los eventos ya son el contrato de integración. El costo del límite se paga hoy (alguna lectura que sería un join se vuelve una llamada al API del módulo o un evento), a cambio de no desenredar contratos invisibles en producción.

2. Tres patrones de arquitectura y justificación

2.1 Hexagonal (Ports & Adapters)

El dominio (pólizas, riesgos, reglas de negocio: una individual solo tiene 1 riesgo, no se renueva una póliza cancelada, la cancelación propaga a los riesgos, prima = canon × vigencia) vive aislado de la infraestructura. Los puertos son las interfaces que el dominio necesita: persistencia, envío al CORE, publicación de eventos, proveedor de notificaciones. Los adapters son las implementaciones concretas: JPA/PostgreSQL, cliente HTTP hacia WebLogic, event bus interno, SMTP/proveedor SMS.

Por qué aquí: la migración declarada a microservicios y el swap del event bus interno por Kafka dependen de que infraestructura y dominio estén desacoplados. Con puertos, ese swap es un adapter nuevo; sin ellos, es una reescritura. Además, las reglas de negocio son el corazón de la prueba: hexagonal las mantiene testeables sin levantar infraestructura.

2.2 Event-Driven (con Outbox)

Los tres procesos que el enunciado exige son de naturaleza de eventos: notificaciones de creación/renovación, renovación automática por expiración de vigencia, y sincronización con el CORE. La API es síncrona en el borde y dirigida por eventos en el interior. El cambio de estado y el evento se persisten en la misma transacción (outbox), lo que elimina la dual-consistencia entre "lo que guardé" y "lo que reporté al CORE".

Por qué aquí: acoplar la respuesta HTTP a la latencia del CORE legado degrada la API y el 24/7. Con outbox, el cliente siempre obtiene respuesta rápida y consistente con lo persistido; el CORE se sincroniza eventualmente, con garantía de entrega via tracking (`PENDING / DELIVERED / FAILED`), reintentos exponenciales con idempotencia-key, DLQ para fallos permanentes y reconciliación periódica.

2.3 API Gateway (borde único)

Aunque un monolito puede funcionar sin gateway, aquí es el borde único de entrada: exige el header `x-api-key`, ruterea `/v1` (y mañana `/v2`), aplica rate limiting, balancea réplicas con health checks (parte del 24/7) y observa el tráfico de punta a punta.

Usamos un API Gateway para que todas las solicitudes entren por el mismo lugar. Ahí validamos la API key, controlamos el tráfico y enviamos cada solicitud a donde corresponde. Esto evita repetir esas tareas en cada módulo. Si más adelante separamos los módulos en servicios, podemos cambiar a dónde se dirige cada solicitud sin pedirles a los clientes que cambien la forma de conectarse.

3. Modelo de datos principal

Modelo relacional sobre PostgreSQL, también para las pruebas locales mediante Docker. H2 se reserva exclusivamente para la implementación simplificada del Módulo 2, que se ejecuta sin infraestructura externa; no forma parte de esta propuesta de arquitectura. Una instancia, un schema por módulo, sin joins ni FKs cruzados entre schemas: los módulos se comunican por API/eventos, nunca accediendo a tablas ajenas. Así, extraer un módulo como servicio mañana equivale a mover su schema, sin remodelar datos. Las reglas de negocio viven en el backend; la base conserva solo constraints estructurales como última línea de defensa (FK, NOT NULL, CHECK de estados).

Excepción deliberada: `Risk` comparte el schema de `Policy`. Forman un único agregado transaccional — la regla "cancelar póliza cancela todos sus riesgos" exige atomicidad inmediata — y separarlos obligaría a coordinar un saga para algo que es una transacción local.

Conexiones: un único pool (HikariCP) sirve a toda la instancia; una conexión es a la base de datos, no al schema, así que el schema viaja calificado en cada sentencia (`@Table(schema = ...)` + Flyway con historial por schema). La prohibición de acceso cruzado se aplica con un test de arquitectura (ArchUnit). Un pool por datasource aparece al extraer un módulo a su propia BD, o antes solo si un módulo congestiona el pool — viable porque no existen transacciones cruzadas entre módulos.

Entidades

- Policy (una sola tabla `policy`) — id UUIDv7, policy_type_id (FK al catálogo),

estado (`ACTIVE`, `RENEWED`, `CANCELLED`), vigencia (inicio/fin), canon mensual (`monthly_rent`), prima (`premium` = canon × meses de vigencia), timestamps de auditoría y `@Version`. Entidad independiente: sin herencia JPA.

- PolicyType — tabla catálogo (`1=INDIVIDUAL`, `2=GROUP`) con descripción,

referenciada por FK desde `policy`. Los ids del catálogo son fijos (secuenciales).

- Identificadores del dominio: UUIDv7 (RFC 9562, time-ordered) — las PK de

`policy` y `risk` son UUID generadas por Hibernate (`@UuidGenerator`, style TIME) antes del INSERT: no adivinables desde el API (mitiga enumeración/IDOR), no filtran volumen de negocio, y se generan en cualquier nodo sin coordinar con la DB (apoya la extracción a microservicios). Las tablas de infraestructura (`outbox_event`, `ipc`, `notification`) conservan BIGINT secuencial: son internas y su orden ayuda al worker. Las referencias hacia el dominio (`policy_id`) son UUID.

- Regla por tipo en el service — la diferencia entre tipos (individual: máx 1

riesgo; colectiva: N riesgos) es regla de negocio, no jerarquía de clases ni estructura de almacenamiento distinta.

- Risk — tabla propia relacionada a la póliza (FK), con su propio estado

(`ACTIVE`/`CANCELLED`) y datos del riesgo asegurado. La regla "una individual solo tiene 1 riesgo" y "solo colectivas agregan riesgos" se aplica en el service.

- OutboxEvent — tabla de integración: id, tipo de evento (`ACTUALIZACION`,

`CREACION`, `RENOVACION`), payload, estado (`PENDING`/`DELIVERED`/`FAILED`), intentos, idempotency-key, timestamps. Se escribe en la misma transacción que el cambio de estado; el worker la drena.

- Ipc — tabla de soporte de la renovación: fecha de aplicación y porcentaje.

Permite auditar qué IPC se usó en cada renovación y configurar el próximo.

- Notification — tracking de notificaciones: canal (`EMAIL`/`SMS`),

destinatario, evento origen, estado de entrega, intentos.

Relaciones

```
Policy 1 ■■■■ N Risk
Policy 1 ■■■■ N OutboxEvent (evento de sincronización al CORE)
Policy 1 ■■■■ N Notification
Ipc           (referenciada por fecha de aplicación en la renovación)
```

4. Pilares no funcionales

4.1 Escalabilidad

En este dominio dominan las lecturas (el front consulta pólizas y sus riesgos frecuentemente; las escrituras son transacciones acotadas). Estrategia:

- Réplicas horizontales stateless del monolito detrás del gateway/balanceador.
- Caché de lecturas frecuentes (`GET /polizas`) con TTL corto e **invalidación al

escribir** (crear/modificar/renovar/cancelar invalida la clave afectada).

- Read replicas de la base como fase posterior (CQRS-lite), no en la prueba: el

volumen del caso no lo justifica y añade retención de consistencia.

- El estado nunca vive en memoria del proceso, por eso escalar es añadir réplicas.

Cómo se manejan y "sincronizan" las réplicas. Las réplicas de la aplicación no se sincronizan entre sí: comparten el estado vía la base de datos y el caché — cualquier réplica puede atender cualquier petición (sin sesiones HTTP; auth por `x-api-key`). Los procesos periódicos se coordinan con locks en la BD: el cron de renovación usa ShedLock (una sola réplica ejecuta por tick) y los workers del outbox son competing consumers con `FOR UPDATE SKIP LOCKED` (cada evento lo procesa una réplica, sin coordinación). Operativamente: balanceador con health checks (`/actuator/health`), auto-restart en Kubernetes, HPA por CPU/RPS, config idéntica por variables de entorno. Las réplicas de lectura de la base sí se replican por streaming replication: la primaria escribe su WAL y las réplicas lo aplican en continuo (asíncrono, lag de milisegundos) — escrituras 100% a la primaria, lecturas a réplicas, con regla de read-your-own-writes (tras una escritura, la lectura de esa póliza va a la primaria). Para las pruebas locales, las réplicas de aplicación comparten una instancia de PostgreSQL en Docker. Si se quiere probar también la replicación de la base, se levantan contenedores de primaria y réplica con streaming replication antes de ejecutar k6.

4.2 Logs y observabilidad

- Logs estructurados (JSON) con campos fijos: `timestamp`, `nivel`, `requestId`,

`resumen corto`. Nada de texto libre ni payloads completos: los logs del flujo al CORE registran el intento y el resultado, no el body.

- Correlation id de punta a punta: nace en la petición de API y fluye

API → outbox → worker → CORE → notificación (viaja en el payload del evento). Así se rastrea una renovación individual desde la petición HTTP hasta el correo enviado.

- Métricas (Micrometer/Prometheus): eventos outbox pendientes (backlog), tasa de fallo de entrega al CORE, estado del circuit breaker, latencia del CORE, tasa de renovaciones automáticas, fallos de notificación.
- Alertas sobre esas métricas: el caso "CORE rechaza un evento por datos inválidos" no se mitiga con reintentos (es fallo permanente) sino con DLQ + alerta + hotfix del contrato, respaldado por contract tests contra el CORE que detectan el drift de contrato antes de producción.

4.3 Tolerancia a fallos

- Circuit breaker en el cliente hacia el CORE (capa media WebLogic): si el CORE falla, se abre el breaker y entra el modo degradado.
- Modo degradado: la API sigue operativa al 100% para el cliente — las pólizas se crean, modifican, renuevan y cancelan localmente; sus sincronizaciones quedan en `PENDING` en el outbox y el cliente recibe la respuesta normal con aviso de estado degradado. Al recuperarse el CORE, el worker drena el outbox con reintentos exponenciales e idempotency-key (el CORE puede recibir duplicados lógicos, no efectos).
- Fallo del proveedor de notificaciones: reintentos + DLQ + alerta; el tracking en la tabla `Notificacion` permite reenvío controlado.
- Fallos permanentes (ej. 400 del CORE): nunca reintento infinito; van a DLQ con alerta para corrección manual/hotfix de contrato.
- Reconciliación periódica con el CORE para cerrar brechas de sincronización.
- Renovación automática con lock de líder (ShedLock) para que réplicas no la ejecuten en duplicado.

4.4 Concurrencia y saturación de la base

- Lock optimista (`@Version`) en `Policy` y `Risk`: transiciones concurrentes (renovar vs cancelar sobre la misma póliza) revalidan la versión al escribir — el perdedor recibe `409`. La contención por fila es baja en este dominio; el lock optimista convierte la carrera en un conflicto explícito y reintentable.
- Reglas check-then-act atómicas: "una individual solo tiene 1 riesgo" se valida dentro de la misma transacción de la escritura (el update del estado toma el lock de fila y el concurrente revalida), con índice único de respaldo en la BD.
- Cron vs API: la renovación automática (ShedLock) y la manual comparten el mismo `@Version` — no hay doble aplicación.
- El pool acotado es la válvula de saturación: `maximumPoolSize` + `connectionTimeout` hacen que bajo carga extrema las peticiones fallen rápido (`503`) en vez de apilar conexiones que tumban la DB. Rate limiting en el gateway rechaza el excedente antes (`429`). Timeouts en toda la cadena (statement, `@Transactional(timeout=)`, HTTP al CORE) garantizan que nada retenga conexiones indefinidamente.
- Bulkhead (Resilience4j) en el cliente del CORE: su lentitud consume su propio compartimento, no los hilos ni conexiones del borde síncrono. La caché de lecturas absorbe el pico de consultas y el outbox desacopla los picos de sincronización.
- Métricas de HikariCP (conexiones activas, tiempo de espera por conexión) con alerta: la saturación se detecta antes de ocurrir.

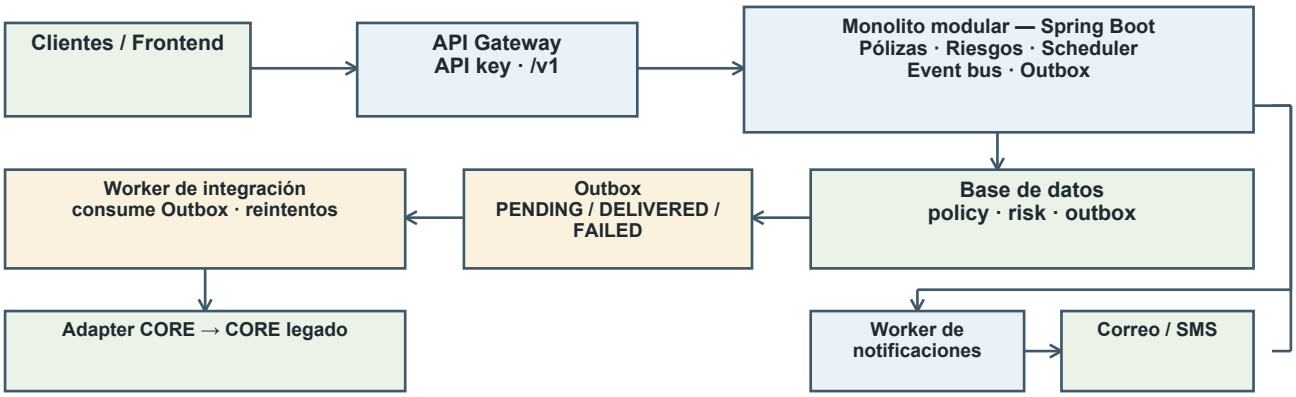
Modelo de capacidad bajo carga legítima (ej. 125k usuarios concurrentes con k6): usuarios concurrentes ≠ RPS — con ley de Little (1 petición por usuario cada ~10 s), 125k usuarios ≈ 12.5k req/s. La carga se sostiene escalando, no bloqueando: réplicas stateless detrás del balanceador (~2.5k RPS por instancia → ~5 réplicas), caché de lecturas con alto hit ratio (deja la DB en ~2.5k req/s), virtual threads de Java 21 (eliminan el techo de 200 hilos de Tomcat), y sizing global del pool (la suma de pools de todas las instancias debe caber en `max_connections` de la base). A ese volumen de lectura se activa el trigger declarado de las réplicas de lectura (CQRS-lite). El rate limiting por usuario no aplica a usuarios legítimos; lo que queda es admisión de sobrecarga (`503 + Retry-After`) y la solución de fondo es escalar. Las pruebas de carga se ejecutan contra

PostgreSQL en Docker, no contra una base embebida.

4.5 Versionamiento de APIs

- Versionamiento por path (`/v1/...`): explícito, fácil de rutear y filtrar en el gateway, y de depurar en logs.
- Criterio de cambio breaking = compatibilidad hacia atrás:
- Añadir un campo opcional a `Policy` no es breaking (los consumidores viejos lo ignoran) → no hay bump de versión.
- Quitar/renombrar un campo, cambiar su tipo o su semántica es breaking → nueva versión `/v2` en paralelo, con ventana de deprecación de `/v1` y headers `Deprecation` / `Sunset` comunicados vía gateway.

5. Diagrama de componentes



Lectura del diagrama: todo el tráfico externo entra por el gateway (único borde con seguridad y versión); los módulos del monolito comparten base de datos y se comunican por el event bus (puerto interno); nada llega al CORE sin pasar por el outbox; las notificaciones y la entrega al CORE son workers desacoplados que escalan y fallan de forma independiente del borde síncrono.

Módulo 2 — Prueba técnica práctica

El código se entrega en GitHub, según el enunciado. El UML está en modulo2_uml_entidades.md; la colección Postman y las instrucciones están en polizas-api/.

Módulo 3 — Optimización de bases de datos

Problema

Se tienen dos tablas de gran tamaño:

- `orders`: 10 millones de registros.
- `customers`: 500.000 registros.

La siguiente consulta busca las órdenes de los clientes de México y presenta un bajo rendimiento:

```
SELECT
    o.order_id,
    o.order_date,
    c.customer_name,
    o.total_amount
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE c.country = 'México';
```

Antes de aplicar cambios revisaría el plan de ejecución con `EXPLAIN ANALYZE` —o su equivalente según el motor— para identificar full table scans, joins costosos, cantidad de filas leídas, CPU, I/O y diferencias entre las filas estimadas y las reales.

Estrategias de optimización

1. Crear índices adecuados

La consulta filtra por `customers.country` y relaciona las tablas mediante `customer_id`. Evaluaría estos índices:

```
CREATE INDEX idx_customers_country_customer
    ON customers (country, customer_id);

CREATE INDEX idx_orders_customer
    ON orders (customer_id);
```

El primer índice ayuda a localizar los clientes de México y sus identificadores. El segundo permite encontrar rápidamente las órdenes asociadas a esos clientes. También verificaría que `customers.customer_id`, al ser clave primaria, tenga un índice.

No crearía un índice por `order_date`, porque esa columna no participa en el filtro ni en el ordenamiento de la consulta.

Los índices aceleran las lecturas, pero aumentan el almacenamiento y el costo de las operaciones `INSERT`, `UPDATE` y `DELETE`, por lo que mediría su impacto.

2. Actualizar estadísticas y revisar el plan

Actualizaría las estadísticas de las tablas e índices para que el optimizador conozca la cantidad y distribución real de los datos. También revisaría la selectividad de `country`.

Si una gran proporción de los clientes pertenece a México, un full table scan podría ser la opción correcta: recorrer un índice y acceder después a muchas filas puede costar más que leer la tabla completa. Por eso no asumiría que todo full scan es necesariamente un error.

También verificaría que no existan conversiones, funciones, espacios o diferencias de formato sobre `country` que impidan utilizar el índice.

3. Reducir los datos procesados y transferidos

Confirmaría que el consumidor necesita todas las columnas seleccionadas. Si alguna no es necesaria, la eliminaría para reducir lecturas y transferencia de datos.

No reescribiría la consulta usando subconsultas o `EXISTS` sin evidencia. La consulta ya es sencilla y el optimizador puede elegir el orden más eficiente del `JOIN`. Probaría una reescritura solo si `EXPLAIN ANALYZE` demuestra que produce un plan mejor.

4. Usar una solución de lectura especializada cuando la consulta sea muy frecuente

Si la consulta se ejecuta constantemente y los datos no necesitan actualizarse de forma inmediata, evaluaría una vista materializada o una tabla de consulta precalculada. Esto reduce el costo de cada lectura, aunque requiere definir cómo y cuándo actualizar la información.

También podría utilizar una caché si se repiten las mismas consultas y los datos cambian poco. Si el problema principal es el volumen de lecturas, consideraría réplicas de lectura para distribuir la carga sin enviar las escrituras a esas réplicas.

El particionamiento o el sharding serían opciones posteriores. El particionamiento tendría sentido si existe un criterio adecuado, como `order_date`; el sharding solo lo evaluaría cuando una única base de datos ya no pueda soportar el volumen, porque introduce mayor complejidad operativa.

Validación

Después de cada cambio compararía el plan y el rendimiento antes y después, usando:

- tiempo de ejecución;
- filas y bloques leídos;
- consumo de CPU e I/O;
- latencia bajo carga;
- impacto de los índices en las escrituras;
- almacenamiento adicional.

Las pruebas deben utilizar datos representativos y diferentes niveles de selectividad para confirmar que la mejora no funciona únicamente para un caso particular.

Conclusión

Comenzaría con `EXPLAIN ANALYZE`, estadísticas actualizadas e índices sobre `customers(country, customer_id)` y `orders(customer_id)`. Luego mediría el resultado. Si la consulta continúa siendo costosa por su frecuencia o volumen, evaluaría caché, vistas materializadas o réplicas de lectura antes de considerar particionamiento o sharding.

Módulo 4 — Incorporar un cambio específico con Git

Problema

Estoy trabajando en `feature/new-login` y un cambio crítico de seguridad ya fue integrado en `main`. Necesito traer solo ese cambio a mi rama, sin incorporar el resto de los commits de `main`.

Estrategia

Usaría `git cherry-pick` sobre el commit que contiene la corrección. Este comando aplica los cambios introducidos por ese commit a la rama actual y crea un commit nuevo en ella. No fusiona toda la rama `main`.

Flujo de trabajo, comando por comando

1. Actualizar la información del repositorio remoto

```
git fetch origin
```

Descarga los commits y referencias actualizadas del remoto `origin`, sin cambiar la rama actual ni modificar los archivos de trabajo.

2. Cambiar a la rama que recibirá la corrección

```
git switch feature/new-login
```

Deja activa la rama de trabajo. `cherry-pick` aplicará el cambio sobre la rama actual, por eso es importante verificar que se está en `feature/new-login` antes de continuar.

3. Verificar que el árbol de trabajo esté limpio

```
git status
```

Confirmaría que no hay cambios locales pendientes que puedan mezclarse con la operación. Si los hay, los guardaría en un commit propio o los apartaría antes de continuar.

4. Encontrar e inspeccionar el commit de seguridad

```
git log --oneline origin/main
git show --stat <hash-del-commit>
```

El primer comando muestra los commits disponibles en la referencia actualizada de `main`. El segundo permite revisar el commit elegido y los archivos que modifica. Antes de aplicarlo confirmaría que contiene la corrección necesaria y que no depende de otros cambios de `main`.

5. Aplicar solo ese commit

```
git cherry-pick <hash-del-commit>
```

Git aplica el cambio a `feature/new-login` y, si termina correctamente, crea un commit nuevo. El commit original permanece en `main`; el commit creado en la rama de trabajo tendrá un hash distinto, aunque represente el mismo cambio.

6. Resolver conflictos, si aparecen

Si Git informa conflictos, primero revisaría qué archivos necesitan atención:

```
git status
```

Resolvería los marcadores de conflicto en los archivos, comprobaría que la corrección de seguridad se conserva y luego marcaría los archivos resueltos:

```
git add <archivo-resuelto>
git cherry-pick --continue
```

`git cherry-pick --continue` finaliza la operación y registra el commit nuevo. Si decido cancelarla y volver al estado anterior al intento:

```
git cherry-pick --abort
```

7. Verificar y publicar la rama

```
git show --stat --oneline HEAD
```

Después ejecutaría las pruebas correspondientes y revisaría el diff. Si todo está bien, publicaría la rama:

```
git push origin feature/new-login
```

Si el repositorio protege las ramas, abriría un pull request para revisión e integración.

Consideración

`cherry-pick` incorpora un commit, no sus dependencias automáticamente. Si la corrección requiere otros cambios que también están en `main`, identificaría e incorporaría esos commits necesarios de forma explícita, o revisaría si conviene otra estrategia. Si el commit seleccionado es un merge commit, se debe elegir su línea principal con `-m`.

Resumen

```
git fetch origin
git switch feature/new-login
git status
git log --oneline origin/main
git show --stat <hash-del-commit>
git cherry-pick <hash-del-commit>
# Resolver conflictos, si los hay; luego:
git add <archivo-resuelto>
git cherry-pick --continue
# Ejecutar pruebas y revisar el cambio
git push origin feature/new-login
```

Módulo 5 — Liderazgo técnico y gestión

Caso

Lidero un equipo de ocho desarrolladores. Actualmente, el 40 % de la deuda técnica se concentra en servicios clave, hubo 10 incidentes críticos durante el último mes y el negocio espera una nueva funcionalidad en tres semanas. Además, no existen estándares consistentes, las revisiones de código son irregulares y dos desarrolladores junior tienen brechas técnicas importantes.

1. ¿Cuáles serían mis cinco prioridades en las primeras dos semanas?

Yo haría lo siguiente:

1. Analizaría los incidentes críticos para identificar sus causas, servicios afectados y patrones recurrentes. Primero atendería los riesgos que todavía puedan afectar a los usuarios o comprometer la seguridad.
2. Revisaría la deuda técnica de los servicios clave y priorizaría la vinculada con incidentes, riesgos de seguridad o bloqueos para la entrega. No intentaría resolver toda la deuda de inmediato.
3. Aclararía con negocio el alcance de la funcionalidad que debe entregarse en tres semanas. Separaría lo imprescindible de lo que podría entregarse después.
4. Acordaría prácticas mínimas de calidad para todo el equipo, incluyendo revisión de código, pruebas y criterios de terminado.
5. Conversaría con los dos desarrolladores junior para entender sus brechas y acordar un plan de apoyo con mentoría, tareas graduales y objetivos verificables.

2. ¿Cómo organizaría al equipo para mejorar la velocidad y la calidad?

Organizaría el trabajo según las prioridades acordadas: estabilizar los servicios más problemáticos y entregar primero el alcance esencial de la funcionalidad. Asignaría responsables claros, limitaría el trabajo simultáneo y haría visibles las dependencias y los bloqueos.

También reservaría capacidad para corregir las causas recurrentes de incidentes, en lugar de posponer indefinidamente la deuda técnica. Para las tareas de mayor riesgo y para apoyar a los juniors, promovería el trabajo en pares y la mentoría. Revisaría el avance con el equipo cada semana y ajustaría la distribución del trabajo según los resultados.

3. ¿Qué métricas implementaría para evaluar el desempeño del área?

Mediría tendencias del equipo y del servicio para identificar problemas y decidir dónde actuar. Seguiría:

- La cantidad y severidad de incidentes, su recurrencia y el tiempo de recuperación.
- El tiempo que tarda un cambio desde que está listo hasta que se entrega.
- La frecuencia de entregas y cuántas requieren corrección o rollback.
- Los defectos que llegan a producción y los resultados de las pruebas.
- El trabajo completado frente al comprometido y el tiempo que permanece bloqueado.
- La evolución de los riesgos técnicos prioritarios.

Primero establecería una línea base y después revisaría si las métricas mejoran. No usaría métricas aisladas, como líneas de código o cantidad de commits, para comparar la productividad individual.

4. ¿Qué prácticas técnicas establecería como obligatorias?

Establecería pull requests para los cambios, revisión por otra persona y revisión de un senior para cambios de alto riesgo. También exigiría pruebas adecuadas al cambio y que los checks de integración continua pasen antes de integrar.

Acordaría estándares de código y una definición de terminado que incluya pruebas y documentación relevante. Procuraría que los cambios sean pequeños y fáciles de revisar, y que las refactorizaciones amplias no se mezclen con una entrega urgente. Para cada incidente importante, registraría las causas y las acciones preventivas, con responsables.

5. ¿Cómo gestionaría la presión del negocio sin comprometer la calidad?

Conversaría pronto con negocio para confirmar el resultado que realmente necesita en tres semanas. Propondría entregar primero un producto mínimo viable, con criterios de aceptación claros, y planificar el resto en entregas posteriores.

Presentaría las opciones y sus consecuencias sobre alcance, fecha y riesgo. No prometería una combinación de alcance y plazo que el equipo no pueda cumplir de manera responsable. Si quedara un riesgo residual, lo explicaría junto con las medidas para reducirlo y acordaría cómo proceder.

Para los desarrolladores junior, definiría expectativas claras, mentoría y puntos de seguimiento. Si no observara progreso sostenido después de brindar apoyo y retroalimentación, conversaría con ellos y seguiría el proceso formal de desempeño correspondiente. No tomaría la salida del equipo como primera medida.